Computer Graphics: Lecture 6

Vertex Attributes and Color Blending

Welcome Back!

Last time we learned how to store our vertex positions in a buffer. This is important because it *scales*.

We learned about how color is perceived.

We learned *why* we store color in RGB…and why it's not enough to represent every possible color we can perceive.

We learned about *gamma correction*.
[What is it, and how does it impact us?]

This time

We're going to understand something very important about the graphics pipeline: interpolation.

We're going to draw a differently-colored triangle that smoothly blends its colors.

We're going to learn how to work with more complex vertex buffers, especially interleaved ones. That means we'll be learning what `stride` means.

We're going to learn an incredibly important concept: what a **vertex attribute** is.

Adding colors

Previously, we stored our triangle data in an array.

It looked something like this:

```
const aSimpleVertexBuffer = new Float32Array([
    -0.75, -0.75, 0,
     0.75, -0.75, 0,
        0,  0.75, 0,
]);
```

Then we loaded it into a buffer using `device.createBuffer`, `.getMappedRange`, and `.set`.

Buffers are flexible

That was a very simple buffer, but first of all, it's a little wasteful: If we know that all our z-values will be zero, we can just load them with 0 in the vertex shader.

But more importantly, buffers let you store multiple pieces of information. More than just position.

Frequently there is a lot of data we want to associate with vertices, but there are lots of custom situations. Like being able to eliminate z, or knowing in advance that most of the faces will be facing the same way (like in a Minecraft chunk)

Attributes

For example, what if we want each vertex to have a color?

Then we need to create a new **attribute** that will store color.

Attributes are pieces of information we want to associate with every vertex in a single draw call (typically one model).

Previously our only attribute was `position`, and it was at `@location(0)`

```
@vertex fn vs(@location(0) position: vec3f)
  -> @builtin(position) vec4f { ... }
```

The vertex shader is required to return a 4 element vector at `@builtin(position)`, so we just copied the input position over.

Attributes (2)

So, if that's how we pull the position attribute, how do we add color?

Answer: add another parameter:

```
@vertex fn vs(
  @location(0) position: vec2f,  //let's make it 2D
  @location(1) color: vec3f
) -> @builtin(position) vec4f { ... }
```

Now, our shader is expecting two piece of information (two "attributes") for every vertex: a position and a color. In this case, both are `vec3f`s, but that isn't required.

But something's missing. What should the vertex shader do with it?

Interstage variables

Variables with `@location` or `@builtin` tags are called **interstage variables**.

They are called that because they contain information that gets passed between stages of the pipeline.

If the vertex shader does not return a variable, then the fragment shader will not be able to read it.

Therefore, the vertex shader needs to return both the position and the color. Then the fragment shader can use the color to compute the color of the pixel.

Structs in WGSL

WGSL doesn't support multiple return values. Instead, we use a `struct`.

```
struct VertexOutput {
  @builtin(position) pos: vec4f;
  @location(0) color: vec3f;
}; // semicolon optional. Most sources I've seen include it.
```

Now we can pass the position and color through the vertex shader:

```
@vertex fn vs(@location(0) pos: vec2f, @location(1) color: vec3f)
-> VertexOutput {
    var vo: VertexOutput;
    vo.pos = vec4f(pos, 1);
    vo.color = color
    return vo;
}
```

`var` in WGSL

This is the first time we've seen `var` in WGSL

WGSL uses `const` for compile-time constants, `let` for readonly variables, and `var` for fully-mutable variables.

In this case, we want to write into our vertex output before returning it, so we need to use `var`.

This is a common pattern in WGSL vertex shaders: instantiate a `var` for an output struct, and then return it.

The fragment shader

The fragment shader takes the struct now:

```
@fragment fn fs(vo: VertexOutput) -> @location(0) vec4f {
    return vec4f(vo.color, 1.0); // convert vec3 to vec4
}
```

Note: `@locations` and `@builtins` can be attached to struct fields. These are both examples of **io-attributes**. These attributes are only meaningful if the struct is an argument or return value of a shader.

If it isn't, the locations are ignored.

By the way: this fragment shader has a bug. We'll see why soon.

Questions?

How to fill the buffer?

So we've made changes to the vertex shader that need to be reflected in the data in our vertex buffer:

1. We've made the input position 2D (because all the z values were 0)[1]
2. We're referring to color at location 1, so that data has to be there.

So now, instead of only having position in our buffer, there's an RGB color value too.

RGB can be stored many different ways. The simplest is one float per channel (we could also pack it into a single integer, but this is easier)

[1] you don't have to do this, I just want to show how flexible vertex shaders are

Interleaving vs SOA vs separate buffers

There are essentially 2 ways we can organize our new buffer.

1. The most common way is *interleaving*. This means we put the different attributes next to each other in order. So position of vertex 0, color of vertex 0, then position of vertex 1, color of vertex 1, then position of vertex 2, color of vertex 2.
2. Alternatively, you can store each attribute in its own buffer.
3. (technically there's a 3rd: we can overload the first buffer but keep each attribute together. this one is kind of a pain)

Interleaving

By far the most common is interleaving. It's cache efficient. Each vertex has all its data sequentially.

```
const vertData = new Float32Array([
//    position      color
//    x     y       r  g  b
    -.75, -.75,     1, 0, 0,   // vert 0
     .75, -.75,     0, 1, 0,   // vert 1
       0,  .75,     0, 0, 1,   // vert 2
]);
```

Let's use this layout for now, and treat the others as exercises.

Loading an interleaved array

Creating the buffer is the exact same as before.

We set its length to `vertData.byteLength`, we map it, and we copy over `vertData` to the mapped range.[1]

```
const buf = device.createBuffer({
    size: vertData.byteLength,
    usage: GPUBufferUsage.VERTEX,
    label: "triangle verts",
    mappedAtCreation: true,
});
(new Float32Array(buf.getMappedRange())).set(vertData);
buf.unmap();
```

[1] I factored the vertex data and the buffer creation into a function in the sample. This will be useful when we add animation later.

Loading an interleaved array

So the data is now mixed so that we have attribute 0 then attribute 1 for vertex 0, then the same layout for vertex 1, etc.

That data is loaded onto the GPU.

However, we need to tell our pipeline that that's how our vertex buffer is going to be laid out.

So now it's time to pay more attention to the "buffers" field in the pipeline description...

Defining the buffer

```
buffers: [{
    arrayStride: 2 * 4 + 3 * 4, // how far to jump between vertices
    attributes: [ // there are 2!
      { // position
        format: "float32x2",    // position is a vec2<f32>, an xy coordinate
        offset: 0,              // the byte of the _first_ position
        shaderLocation: 0       // @location(0) gets its data from this one
      },
      { // color
        format: "float32x3",    // color is a vec3<f32>, an rgb vector
        offset: 2 * 4,          // byte of the _first_ color (after 2 floats)
        shaderLocation: 1,      // @location(1) gets it data from this one
      },
    ]
}]
```

Defining the buffer (2)

Remember that each vertex has a number: an index.

The GPU will multiply that index by stride to get the location of the data for that vertex.

For an interleaved buffer, stride is the size of the vertex.

So if its wants to transform vertex 2, it will multiply $2 \cdot \text{stride}$ to get the byte address of the second vertex.

Remember, each `Float32` is 4 bytes. So the size of the XY coordinates are 8 bytes, and the size of the RGB is 12 bytes. 20 bytes per vertex.

Therefore, vertex 2 starts at byte 40.

Defining the attribute

Once we know how to locate the start of a given vertex, we need to know how to find the location of each individual attribute: its `offset`.

The offset of position is 0, because the vertex starts with position.

The offset of color is 8, because it comes after position's 2 floats.

Comprehension check:
[what if we had put color first, then position?
What would the offsets be?]

Defining the attribute: comprehension check answer

If color came first, its offset would be 0

Then, because color is 3 floats, position would have offset 12, for 12 bytes.

You can specify the attributes in any order you want, but it's common to make position come first.

Defining the attribute (2)

We also need to specify the `format`. There are a bunch of options here, one for each of the primitive datatypes in WGSL.

In our case, we want a vector: `float32x2` for position and `float32x3` for color. These correspond to a `vec2f` and a `vec3f` respectively.

WebGPU requires that vectors of float 32 be aligned to 16 bytes, so every vector smaller than 4 floats will have zero floats after it, so you can actually declare a `@location(0) pos: vec4f` and the z and w coordinate will be 0.

Lastly, we specify which `@location` the attribute corresponds to.

WebGPU only cares about locations

As a general theme, WebGPU does not like to look things up by string.

We chose to store our position variable at `@location(0)`. It does not matter what we name the parameter. These all are fine:

```
@vertex fn vs(@location(0) pos: vec2f, ...)
@vertex fn vertexShader(@location(0) position: vec3f, ...)
@vertex fn vs_main(@location(0) jaborkatron: vec4f, ...)
```

You can even make it a single float:

```
@vertex fn whatever(@location(0) x: f32, ...) //throws y away
```

WebGPU only cares about locations (2)

But suppose we did this in our shader:

```
@vertex fn vs(
  @location(1) pos: vec2f,
  @location(0) color: vec3f,
) ...
```

This would cause position to pull from the color attribute, so the x and y would actually be the r and g in the color. The b would be ignored.

The color attribute would have x and y for r and g, and 0 for b.

To reiterate: WebGPU doesn't care what you name your variables. It cares about matching up locations with attributes

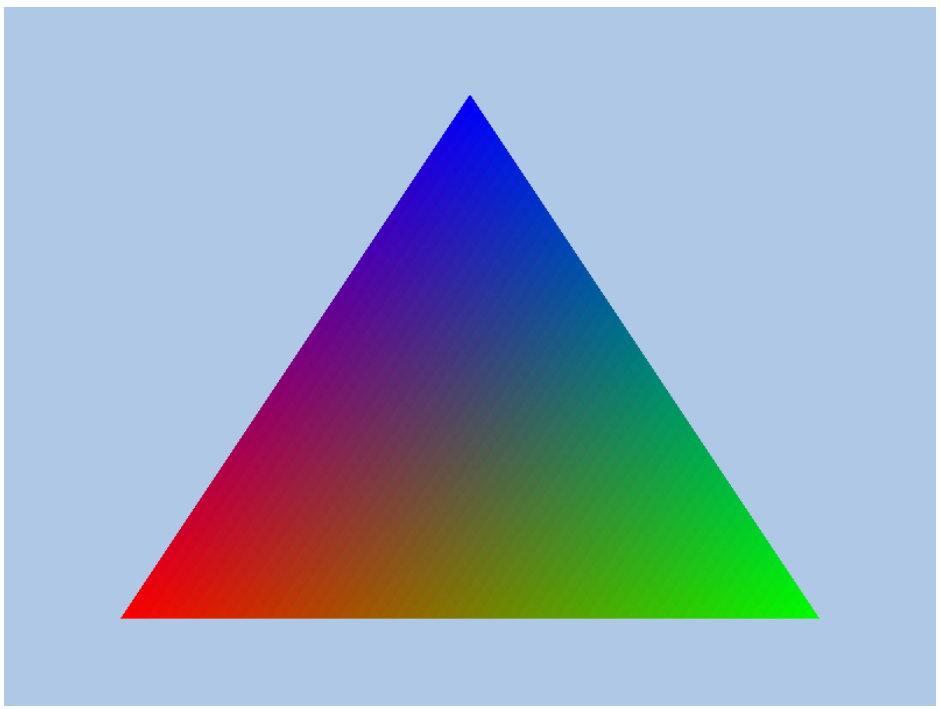
The rest of the sample

We've covered everything that differs from the previous sample.

Try your best to start from scratch, and see how far you can get implementing the sample. Only peek at `sample04` if you get stuck.

The next slide is what you should see if you've been following along…

An erroneously blended triangle



This is the triangle you should see if you followed the slides, but something is wrong with it...

Two questions

You probably have two questions:

1. How did it know to blend the triangle smoothly like that?
2. What is erroneous about it? It looks smoothly blended between red, green, and blue vertices. (in fact: most tutorials don't realize this is wrong, so it will often look like this if you search for a "first triangle" tutorial online)[1]

We're going to answer these questions in order, but feel free to speculate if you have an idea what's wrong with the triangle (but don't spoil it if you read ahead!)

[1]Even my first version of this class didn't realize it is wrong, but it's obvious in retrospect

How did it blend the triangle?

This is one of the most mysterious things about 3D graphics, and also one of the most important. It really forces you to understand the graphics pipeline, including lots of hidden steps.

What's happening is that the fragment shader does not receive the same data as input that the vertex shader returned.

Instead, there is a hidden interpolation step that happened in between.

This happens for *all attributes* by default. **Color is not special**!
Literally every piece of data gets blended like that. Even position!

Recalling the pipeline

- Vertex processing
- Primitive assembly
- Clipping (and w-divide)
- Rasterization
- **interpolation happens here-ish**
- Fragment processing

Rasterization

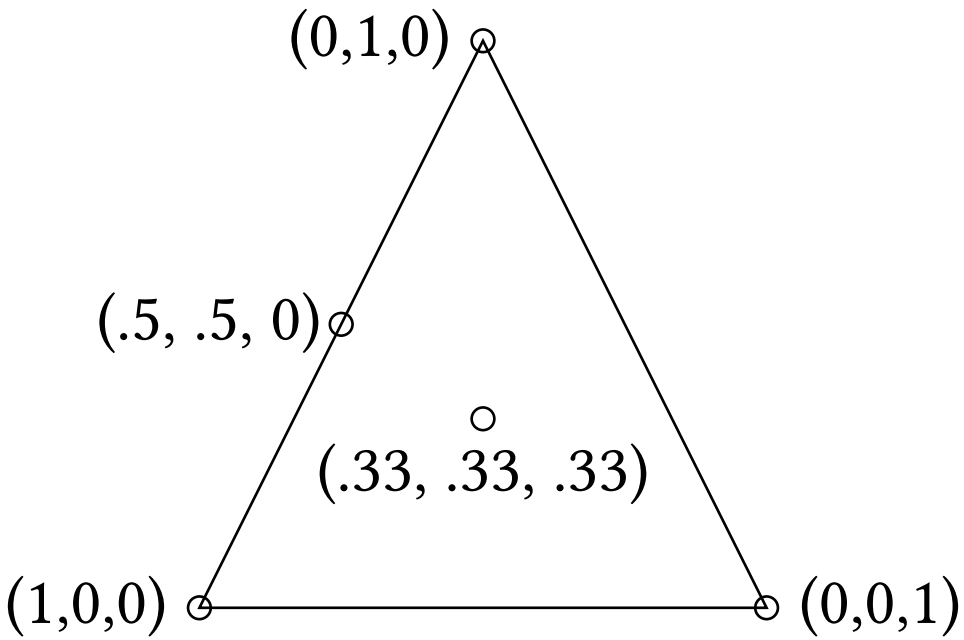
Recall that rasterization is when the GPU determines which pixels are covered by the triangle.

These pixels are in viewport coordinates. That is, instead of using the clipping coordinates (which we've been assuming go from `-1` to `1`), they refer to viewport coordinates, where (20, 20) refers to 20 pixels from the left, and 20 pixels down.

In addition, the rasterizer also computes the **barycentric coordinates** of the fragment on the triangle.

Barycentric coordinates review

Barycentric coordinates are a way to refer to a point on a triangle in terms of how close it is to each vertex.



The barycentric coordinates of some points on a triangle.

Textual description

Put another way, the barycentric coordinates of points on a triangle have three dimensions: one for each vertex of the triangle.

For the 3 vertices of the triangle, the barycentric coordinates are (1, 0, 0); (0, 1, 0); and (0, 0, 1) respectively.

For the point halfway between the first two vertices, there is a 0.5 in the related coordinates (.5, .5, 0).

The point in the center of the triangle has one-third for all three coordinates.

If one of the coordinates is 0, you know the point has to be on an edge (because it's as far as possible from at least one of the points)

If the point is on the triangle, they will sum to 1 (ignoring rounding error).

Calculating barycentric coordinates

We don't actually have to calculate Barycentric coordinates by hand. The GPU will do it for us before the fragment shader is invoked.

How does it do it?

Basically, the dimensions of the barycentric coordinates are the areas of the sub-triangles formed by the point...

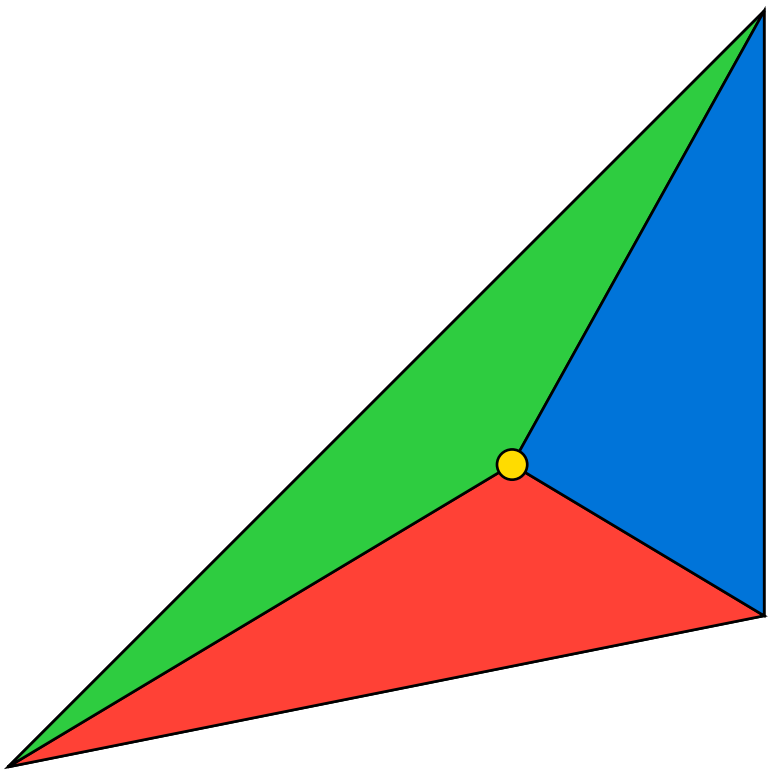
Calculating barycentric coordinates (2)

Notice how any point on a triangle ends up forming 3 sub-triangles.

The barycentric coordinates are the areas of the 3 triangles divided by the area of the whole triangle.

Specifically, the coordinate for the top vertex is the relative area of the bottom (red) area.

How do we calculate the area? The easiest way is to use the two shorter legs of the sub-triangle and compute the area of a parallelogram (which is a 2D matrix determinant).



Why do we care about barycentric coordinates?

The barycentric coordinates are used to interpolate *all of the attributes.*

For example, suppose we have a fragment whose barycentric coordinates are (.5, .5, 0). It is exactly between two vertices.

If one vertex is red (rgb: (1, 0, 0)) and the other is green (rgb: (0, 1, 0)), the actual color value passed to the fragment shader will be (0.5, 0.5, 0).[1]

What if the colors are (1, 1, 0) and (0, 0, 0.5)? Then the final color will be 50% of the first one plus 50% of the second one (and none of the 3rd vertex which we left out). The interpolated color is (.5, .5, .25).

---

[1]For now, this is linear interpolation, because we haven't introduced perspective. Once perspective is introduced, it gets a little more complicated. Perspective is non-linear, so the transformation has to account for that.

Interpolation before fragment processing

So that answers the question of why the triangle is smoothly blended.

All attributes are interpolated by default.[1]

Before the fragment shader is invoked, the rasterizer determines the barycentric coordinates of the fragment.

These coordinates are essentially multiplied by the attributes *returned* by the vertex shader.[2]

---

[1] interpolation can be disabled if you want. It will take the value of the "provoking vertex" (first vertex) instead of interpolating it. The triangle would be a solid color. This can be useful for computing face normals.

[2] again, I'm oversimplifying because we haven't introduced perspective yet.

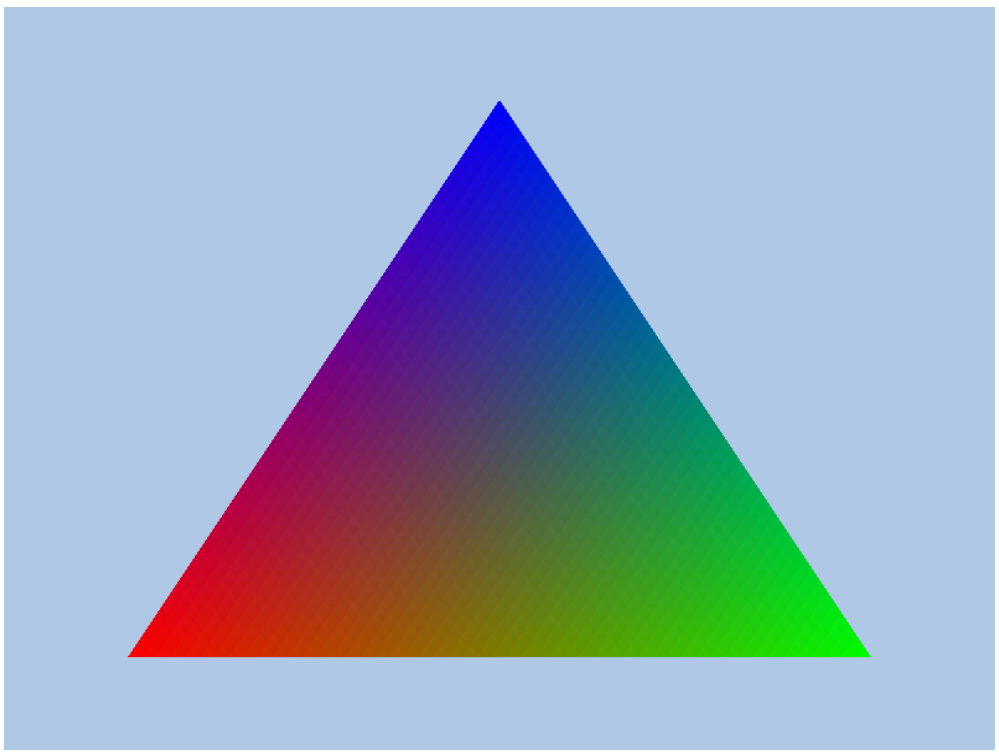
Questions?

Okay, what was wrong with the triangle

Earlier I said there was a problem with the triangle.

Let's consider again: what's wrong with it?

Doesn't it look kind of dark?



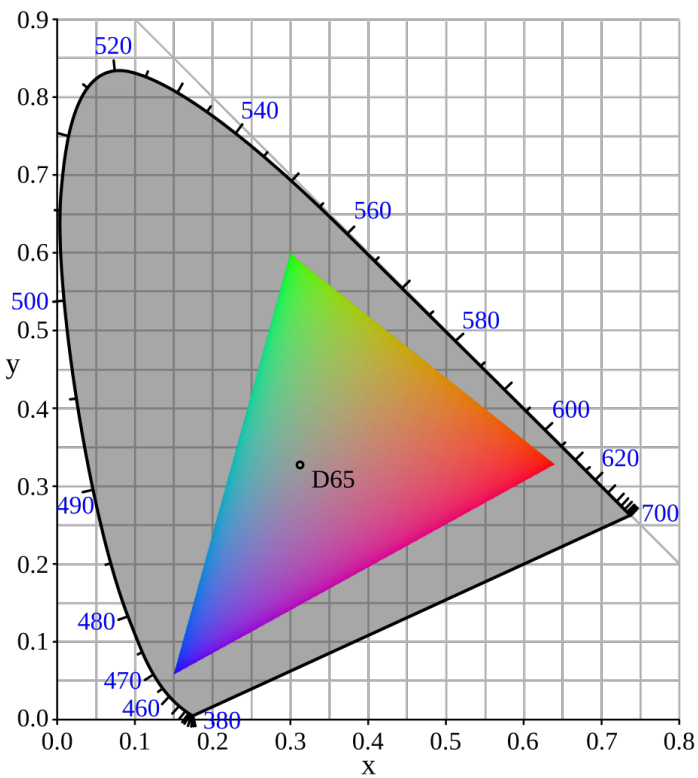
sRGB reminder

Remember the SRGB color space?

We're effectively drawing it. That is, we're covering the entire range between red, green and blue.

Therefore, we expect our result to exactly match the sRGB triangle, but it doesn't!

Forget the order of points, that's easy to fix, but what happened to yellow? And where's light blue? It's too dark!



What's going on?

Time to solve the mystery: the problem is that our fragment shader is not gamma-correct.

The canvas context in WebGPU does not do extra gamma correction for you. It assumes that you will output gamma corrected pixel colors.

One reason for this: almost all images are stored in gamma-encoded values.[1] So if you want to just draw an image or show a movie, you do not want it to do gamma correction again and make it too dark.

---

[1] the reason for this is that to store the raw linear light values would waste lots of bits on small differences in brightness, but use very few bits in the extreme ramp from 0 to 20% brightness. Remember that we perceive a small increase in linear brightness as a large increase in perceived brightness. So it makes sense to break that range down into smaller regions of color.

What's going on?

However, the pixel values have not gone through any kind of gamma correction filter.

If we want to see every color as it appears on the gamut, the raw, linear light values need to be interpolated evenly.

That is happening: the interpolation in the graphics pipeline is interpolating the brightnesses as if they were linear brightnesses, but we're never applying the gamma encoding to the results.

Therefore, it looks too dark. The system is assuming its drawing gamma corrected perceived color, but it's drawing linear color instead. Linear color has smaller values. (e.g., 50% perceived = 20% linear)

How do we fix it?

We have to add gamma correction in our fragment shader output.

Recall that the simple gamma equation is:
$\text{Brightness}_{\text{in}} = (\text{Brightness}_{\text{out}})^{\gamma}$

We know what the input brightness is. We need to solve for output:
$(\text{Brightness}_{\text{in}})^{\gamma^{-1}} = \text{Brightness}_{\text{out}}$

And we typically choose 2.2 for gamma. So, inverse gamma is about 0.45.

So we need to raise our color to that power to adjust it. This will do nothing to maximum bright or dark colors, but mid range values will be larger.

Questions?

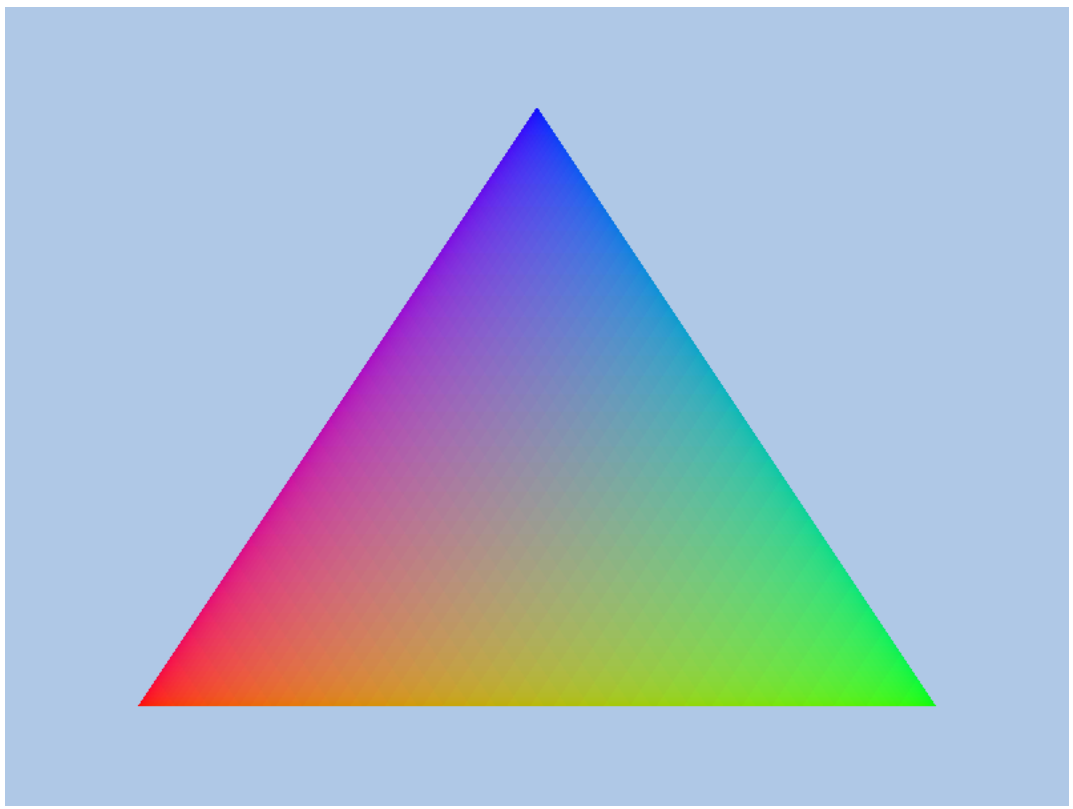
A fixed fragment shader

```
const gamma: f32 = 2.2;
const inv_gamma: f32 = 1.0 / gamma;

@fragment fn fs(vo: VertexOutput)
-> @location(0) vec4f
{
    let perceptual_color =
      pow(vo.linear_color, vec3f(inv_gamma));
    return vec4f(perceptual_color, 1.0);
} // I renamed `color` in the struct to `linear_color`
```

Notice that the `pow` function takes vectors, too. We turn `inv_gamma` into a vector, which just repeats it: `<0.45, 0.45, 0.45>`. The result is each channel being raised to that power.

The correct triangle



Now we're seeing all the colors. Even cyan!

One last adjustment

In our vertex buffer, we're using pure red, pure green, and pure blue, so there is no difference between perceived brightness and linear brightness. 100% to any power is 100%.

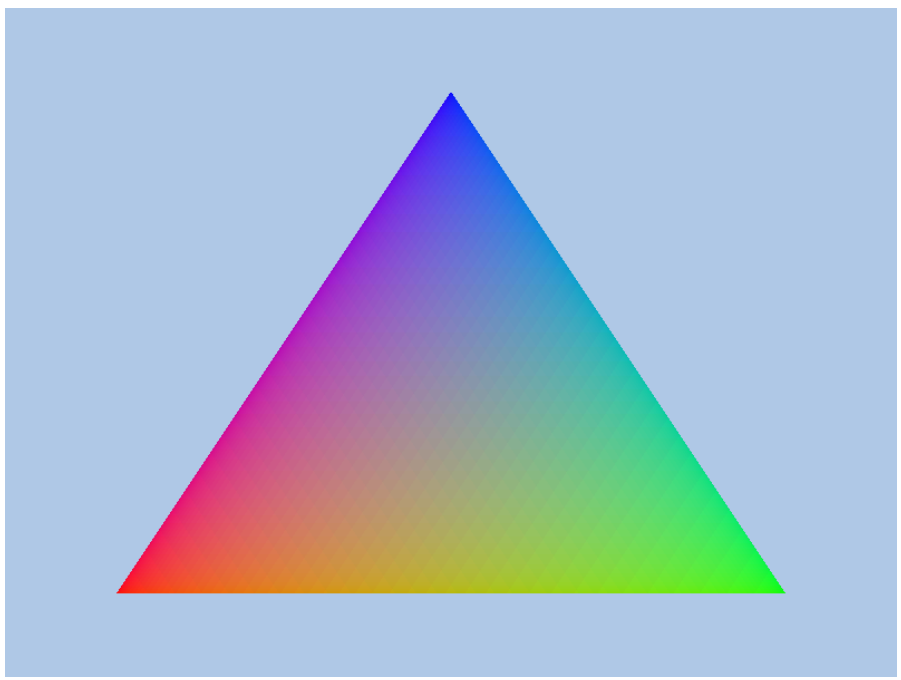
However, if this is a value a human will adjust, we probably want it to be in perceived brightness rather than linear brightness.

This is optional, but it's much easier for humans to intuit perceived color.

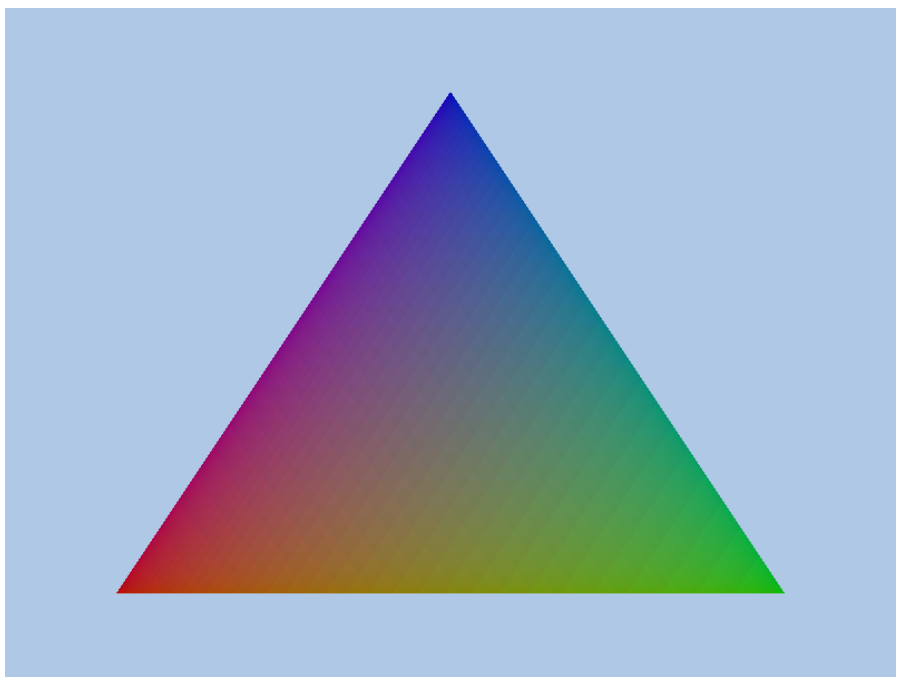
Right now, if we make our vertices 50% red, 50% green, and 50% blue, they will be too bright: they won't be half black, they will be more like 80% bright.

Triangle with linear inputs

I set the brightness to 50%. It doesn't look like it. Looks like 75%…



Fullbright triangle



50% linear brightness

Gamma-corrected inputs

Color is additive in linear space. If we know that one light has brightness X and the other has brightness Y, if we shine them together the combined brightness is X + Y.

Color is **not** additive in gamma space. *Perceptual* brightness is not additive, and we should not linearly interpolate over it.

The automatic interpolation in the pipeline doesn't know anything about gamma correction, it's just interpolating raw vectors based on barycentric coordinates.

Gamma-corrected inputs

In general, we assume that light values are in linear light. We don't use perceptual brightness to define a light.

But color could go either way. If we assume that color refers to perceptual brightness, we need to convert it (by raising it to the power of gamma) first. Then it will be linear.

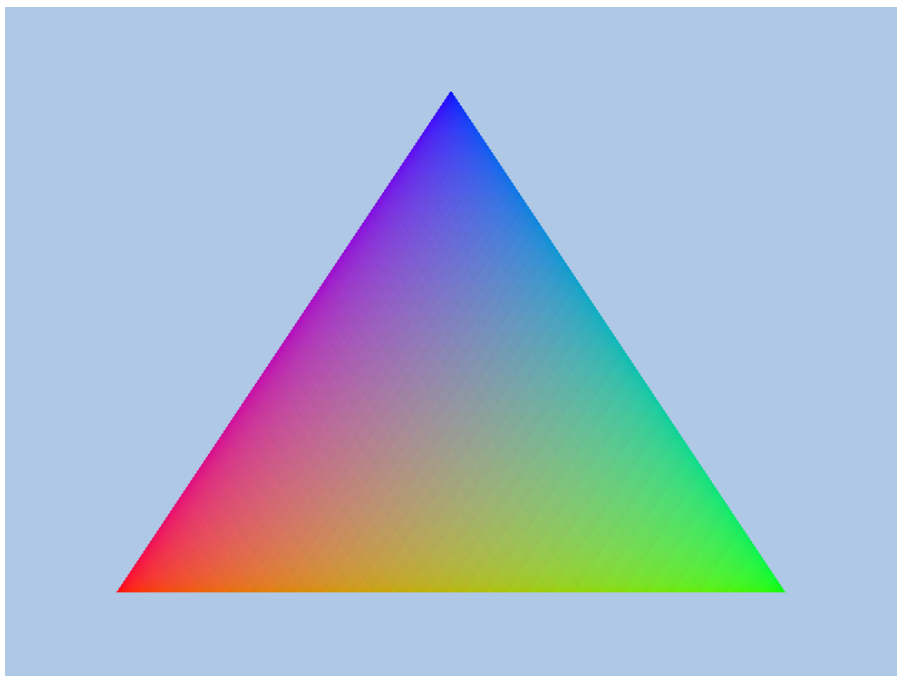
Then, the barycentric interpolation will be correct.

Then, the fragment shader will gamma-encode the linear brightnesses correcly.

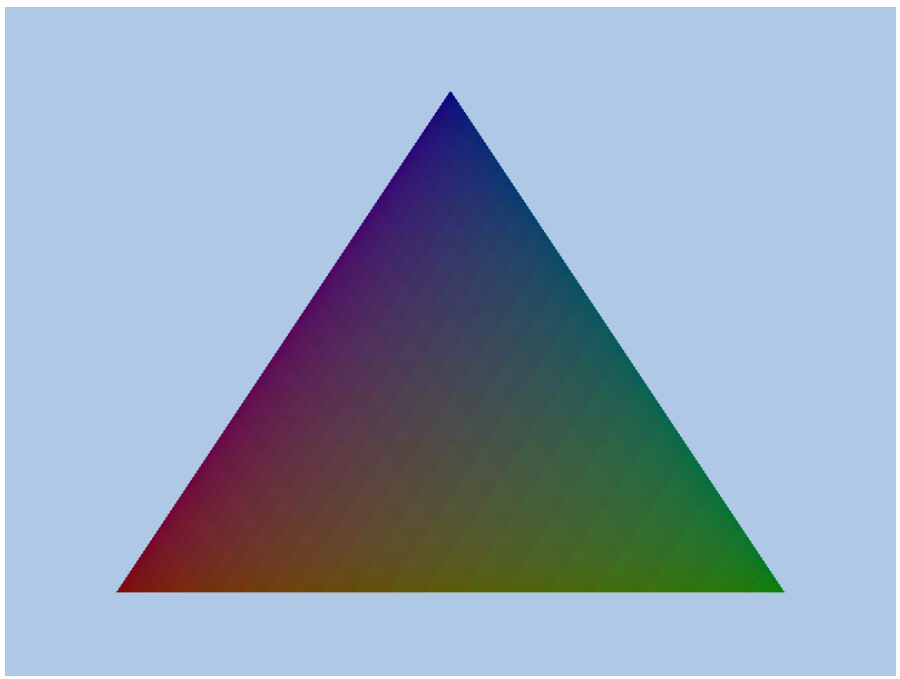
A correct vertex shader

```
@vertex fn vs(
    @location(0) pos: vec2f,
    @location(1) color: vec3f,
) -> VertexOutput
{
    var vo: VertexOutput;
    vo.pos = vec4f(pos, 0, 1);
    vo.linear_color = pow(color, vec3f(gamma));
    return vo;
}
```

The final result



Fullbright triangle



50% perceptual brightness

One more thing still

Still, it's weird to hardcode gamma like that.

What if the user uses a color profile with a different gamma?

And the actual sRGB gamma function is piecewise, so our simple exponent is not exactly right (it's close, but for very dim values it will not match exactly what the profile specifies).

It turns out, the output of the fragment shader can be converted from linear to gamma corrected automatically.

Automatic gamma adjustment

There are 3 things we need to do:

1. Make our canvas format support srgb
2. Make our pipeline render to an srgb format
3. Make our render attachment render to an srgb view.

If we do all three of these, the output of our fragment shader will be assumed to be linear, and the device will convert it to gamma correct automatically and correctly.

Changing the canvas format

We need to update our `initWebGpu` function:

```
context.configure({
device: device,
format: gpu.getPreferredCanvasFormat(),
viewFormats: [
    (gpu.getPreferredCanvasFormat() as string + '-srgb') as GPUTextureFormat
],
colorSpace: "srgb",
alphaMode: "opaque",
});
```

We're taking the existing format and adding `-srgb`. This will give us 2 formats that we can use: one linear and one srgb.

`as` is the keyword for raw casting in Typescript.

Changing the pipeline

We add this same format to our fragment shader target in the pipeline description:

```
targets: [{ format:
  (context.getCurrentTexture().format as string +'-srgb') as
      GPUTextureFormat,
}]
```

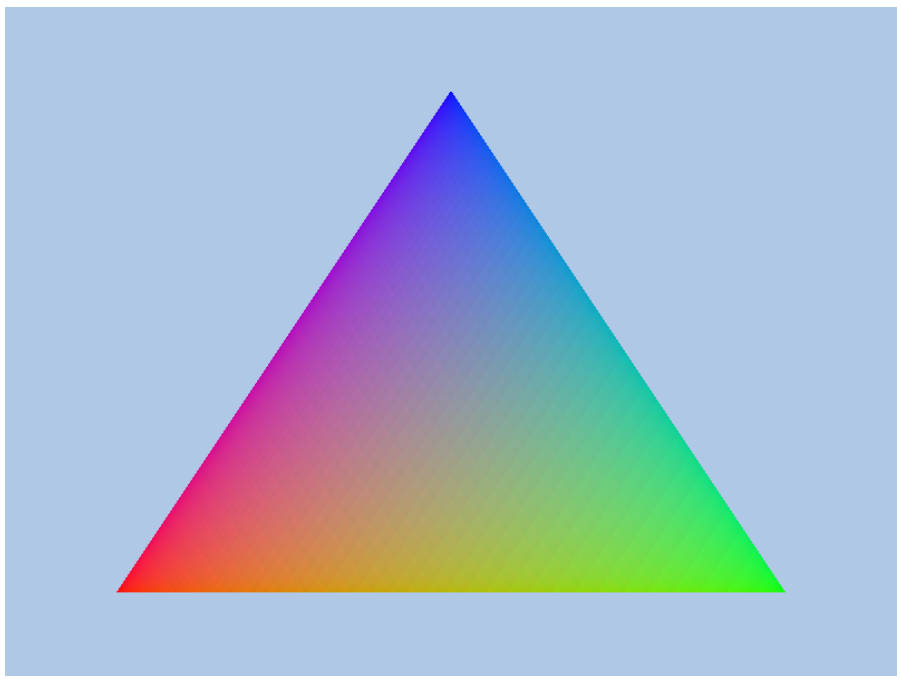
Same as before. We're saying "take whatever format the context is using, but add -srgb to the end of it".

Changing the attachment

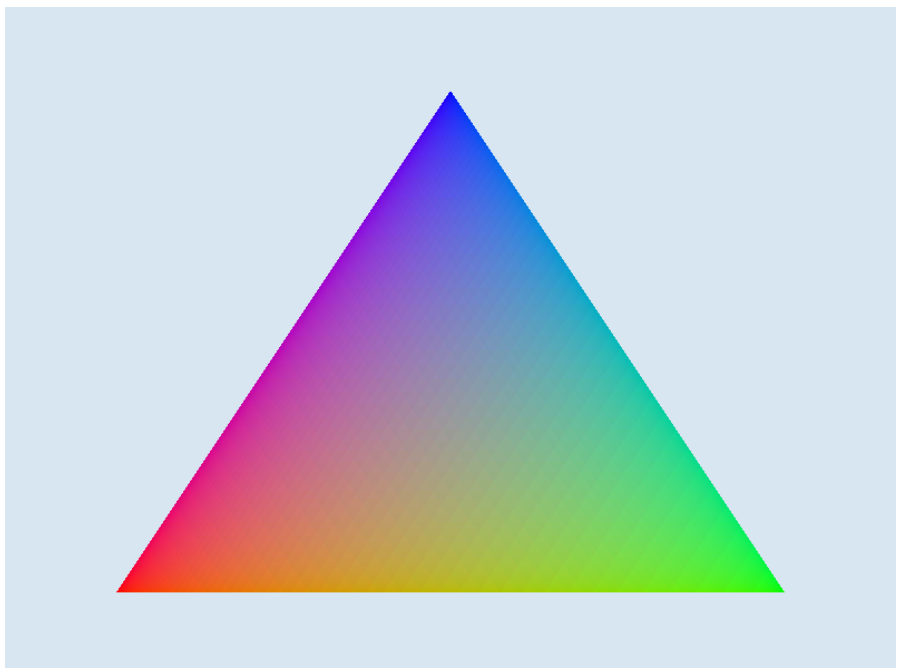
Finally, we create a texture view when rendering. A texture view is basically a little adapter that lets you pretend a texture has a different format. The device will notice that the view wants to be sRGB and convert the results:

```
colorAttachments: [{
    loadOp: 'clear',
    storeOp: 'store',
    view: context.getCurrentTexture().createView(
        {format:
            (context.getCurrentTexture().format as string + '-srgb') as
                GPUTextureFormat
        }),
    clearValue: {r: .7, g: .8, b: .9, a: 1},
}]
```

Manual vs auto gamma comparison



Manual gamma triangle



Auto gamma triangle

I think the auto one looks slightly more saturated and better. YMMV.

Questions?

Summary

Today, we learned a lot:

- How to add multiple attributes
- How to interleave them
- How attributes are interpolated between shader stages
- How barycentric coordinates work
- Some more WGSL (the `pow` function, and how it can work piecewise on vectors)
- How gamma correction applies to color blending

Comprehension checks

- Can you guess how to use 2 buffers instead of one? One buffer would have position and one would have colors. If it's not clear, I recommend trying it out. Notice that the "buffers" field of the pipeline description object takes an array.
- What do you think would happen if you used a smaller gamma value, like 1.5? What about a bigger one, like 3? Come up with a hypothesis before testing it!
- Can you modify the vertex buffer so that it stores 4 elements for the position and 4 for the color? How would you change the buffer data and the format?

Recommended reading

If you're still struggling to understand gamma correction, make sure to read these recommended sources:

- What every coder should know about gamma (and take the quiz)
- Gamma Error in Picture Scaling this one shows how common the mistakes are.

The book covers today's lesson in chapters 1.04 and 1.05. It does not include gamma correction, which may be an oversight, so I opened an issue.

Next time we'll talk about textures, so I recommend reading up to that point in the book. Textures are usually already gamma corrected.

Questions?